💸 KharchaSaathi

Agentic AI WhatsApp Expense Settler for Groups

“Splitwise ki jagah nahi, Splitwise se pehle ki soch — expense reason karo, likho mat.”

---

1. Problem Statement — Kya Solve Kar Rahe Hain

Roommates, hostel groups, trip friends — sabke saath ek hi recurring pain hai:

- Kisi ko yaad rakhna padta hai “kisne kya kharcha kiya”
- Split karne ke liye app kholo, form fill karo, categories choose karo, members select karo
- Mahine ke end mein confusion: “kisko kitna dena hai, kisko kitna lena hai”
- Existing apps (Splitwise, SplitWise clones) **reactive record-keepers** hain — woh sirf data store karte hain jo tum manually feed karte ho. Woh kabhi khud nahi batate “isse pehle ye settle kar lo” ya “tumhara pattern change ho gaya hai”

Root problem asli mein UX friction + zero proactive intelligence hai, feature-gap nahi.

Kaun use karega

- Hostel/PG roommates jo daily kharcha share karte hain
- Trip/travel groups
- Flatmates jo monthly bills split karte hain

Success metric (honest)

- Kya log actually WhatsApp pe likh ke expense log karte hain, ya app khol ke form fill karna prefer karte hain? — Yehi hypothesis test karna core goal hai, sirf feature-building nahi.

---

2. Ye Existing Apps Se Alag Kaise Hai — Honest Comparison

Dimension	Splitwise (existing)	KharchaSaathi
Interface	Dedicated app, forms	WhatsApp — jahan user already hai, koi naya app install nahi
Input	Structured (select payer, select members, enter amount)	Natural language: *“maine 800 kharch kiya trip mein, Rahul aur mujhe split kar do”*
Ambiguity handling	User khud resolve karta hai (dropdown select)	Agent khud resolve karne ki koshish karta hai; agar unclear hai to confirm poochta hai (WhatsApp reply YES/NO)
Bill entry	Manual typing	Photo bhejo, OCR se line-items auto-extract + auto-split
Voice	Nahi	Voice note se expense log
Behavior	Purely reactive (jo daalo wahi milta	Proactive nudges — patterns detect

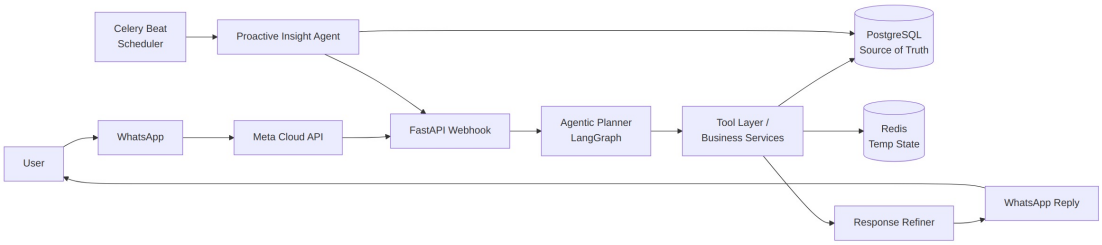
	hai)	karke khud message bhejta hai
Settlement	Tum khud calculate karte ho kise kitna dena hai	Agent automatically minimum-transaction settle-up compute karta hai (graph optimization)

Honest baat: core ledger/split *concept* naya nahi hai — Splitwise ye kar chuka hai. Differentiator ye hai: 1. **Zero-friction natural language interface** (WhatsApp, no new app) 2. **Agentic reasoning layer** — ambiguity resolution + multi-step tool execution, sirf CRUD nahi 3. **Proactive intelligence** — agent khud insight/nudge generate karta hai, user se pooche bina 4. **Debt-graph minimization as a real algorithm**, not just a stored balance

Interview mein yehi answer dena — “main naya ledger concept invent nahi kar raha, main ek agentic reasoning + zero-friction interface layer bana raha hoon jo existing problem ko differently solve karta hai.”

---

3. High-Level Architecture

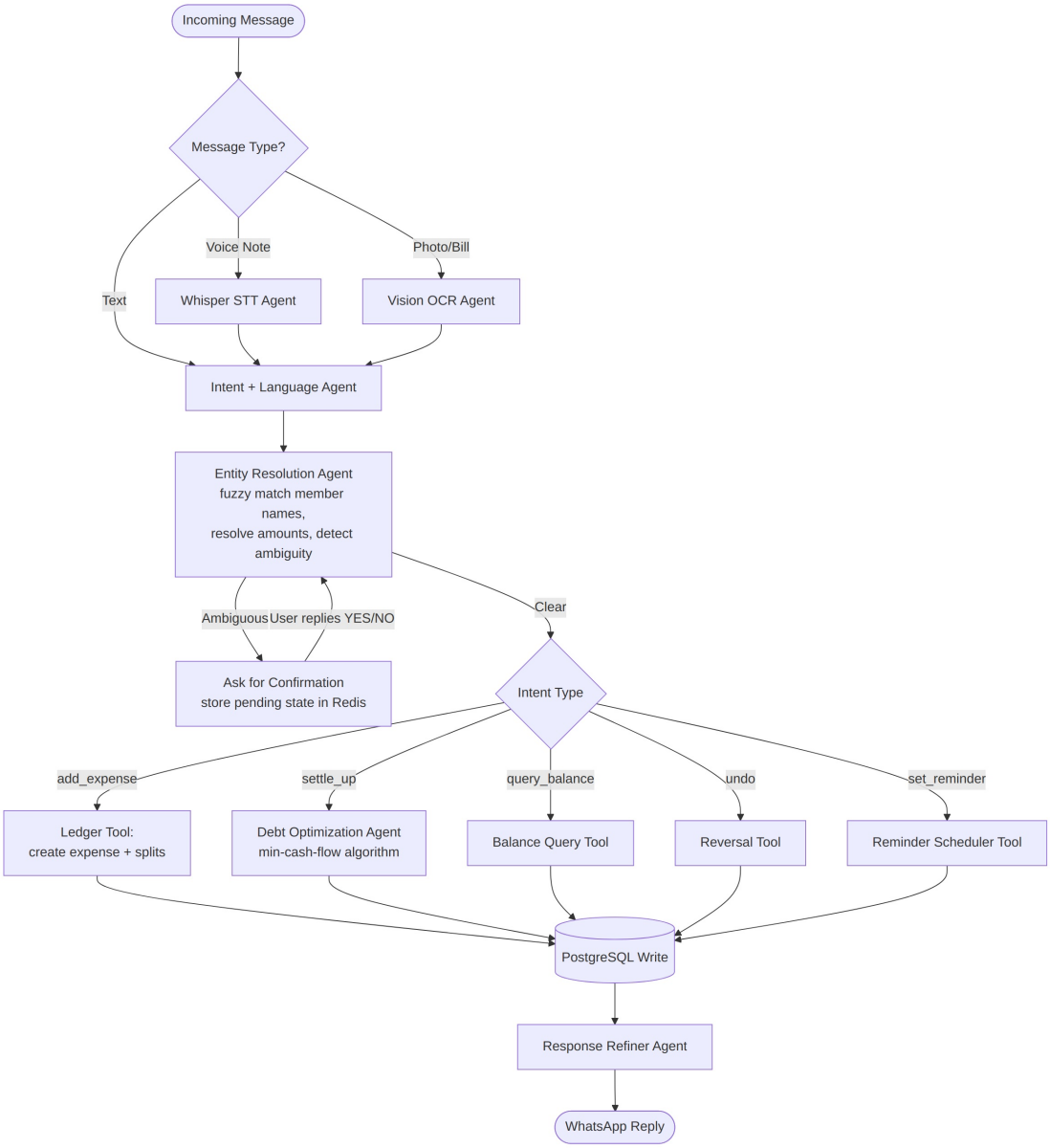


diagram

---

4. Agent Pipeline — Kaun Kya Karta Hai

Ye VyaparAI jaisa multi-step planner hai, lekin split-domain specific logic ke saath.



diagram

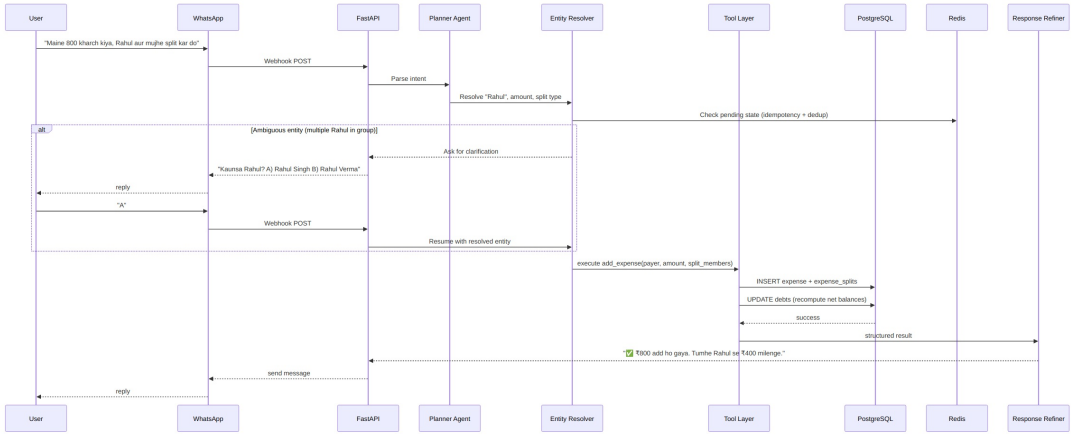
Agent	**Calls**	**Kaam**
Language/Intent Agent	1	Language detect + intent classify (add_expense, settle, query, undo, remind)
Entity Resolution Agent	1	Member fuzzy-match, amount extraction, ambiguity detection
Vision/OCR Agent	1 (conditional)	Bill photo se line-items + amount extract
Voice Agent (Whisper)	1 (conditional)	Voice note transcription
Debt Optimization Agent	0 (pure algorithm, no LLM)	Minimum-transaction settle-up calculation — **yeh pure DSA hai, LLM call nahi**
Proactive Insight Agent	1 (scheduled)	Spending pattern detect, nudge generate

Response Refiner	1	Natural, structured WhatsApp reply banata hai

Interview point: Debt Optimization Agent LLM-based nahi hai — ye deliberate design choice hai. Financial calculation kabhi bhi LLM se nahi karni chahiye (hallucination risk), isliye ye pure deterministic algorithm hai jo agent pipeline mein ek tool ki tarah call hota hai. Ye distinction interviewer ko dikhati hai ki tumhe pata hai kab LLM use karna hai aur kab nahi.

---

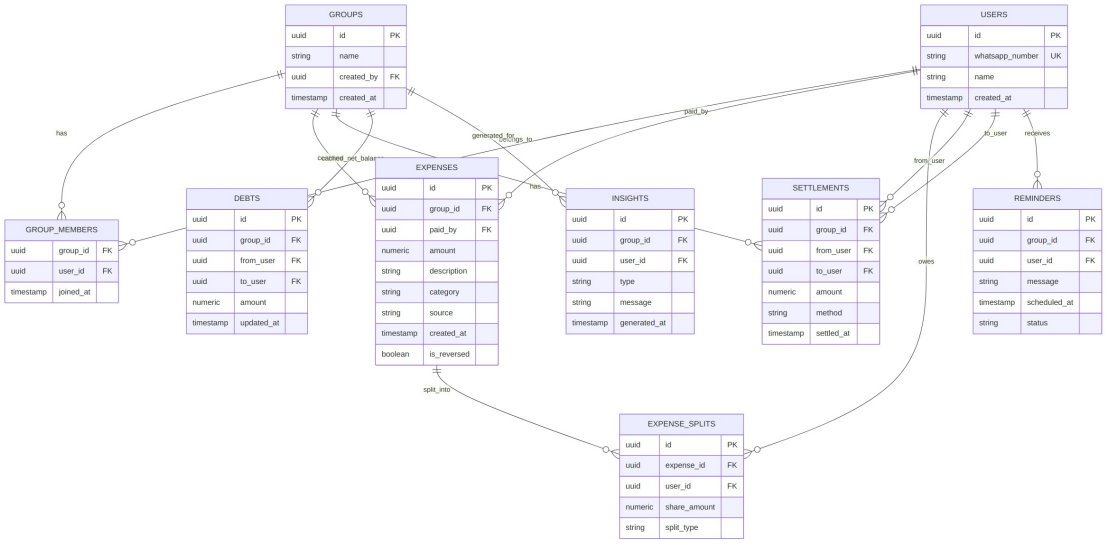
5. Data Flow — Full Request Lifecycle (Sequence Diagram)



diagram

---

6. Database Architecture (PostgreSQL — Source of Truth)



diagram

Design note: `DEBTS` table ek **cached/derived table** hai — asli source of truth `EXPENSE_SPLITS` aur `SETTLEMENTS` hai. Har naye expense/settlement ke baad debts table recompute hota hai (event-driven, not on every read) — ye read-heavy query pattern (balance check bahut frequent hoga) ko fast rakhta hai without recalculating from scratch har baar.

---

7. Redis — Temporary State Layer

Key Pattern	Data	TTL	Purpose
`pending_confirm:{user_id}`	Ambiguous entity resolution context	5 min	Multi-turn confirmation (YES/NO) resume karne ke liye
`msg_dedup:{whatsapp_msg_id}`	Boolean flag	24 hr	Meta webhook retries se duplicate processing rokna (idempotency)
`undo_context:{user_id}`	Last transaction reference	10 min	"undo" command ke liye last action ka reference
`session_lock:{group_id}`	Lock flag	30 sec	Concurrent expense writes same group mein race condition avoid karne ke liye

Interview point — idempotency: WhatsApp/Meta webhooks kabhi kabhi duplicate deliver karte hain (at-least-once delivery guarantee). Agar dedup layer nahi hoga, to same expense do baar log ho sakta hai. Ye production-grade distributed systems problem hai — genuinely interview mein deep-dive karne layak.

---

8. Debt Optimization Algorithm — Core Hard Engineering Piece

Problem: Group mein N logo ke beech multiple IOUs hain. Agar naively settle karo (jisne udhaar liya wahi wapas kare), to transactions bahut zyada ho jaate hain. Goal: **minimum number of transactions mein sabka net balance zero karna.**

Approach: 1. Har user ka **net balance** compute karo: `total_paid - total_owed` 2. Positive balance wale = creditors (paisa lena hai), negative = debtors (paisa dena hai) 3. Greedy matching: sabse bada creditor + sabse bada debtor ko match karo, jitna ho sake settle karo, repeat 4. Ye **max-heap based greedy algorithm** hai — O(n log n) time complexity

```
 Example:
A paid 900, owes 300 → net +600
B paid 0, owes 300 → net -300
C paid 0, owes 600 → net -600
D paid 300, owes 0 (over-contributed) → net +300

Naive: 3 people owe A/D separately = up to 3-4 transactions
Optimized (greedy):
  C pays A ₹600 (settles C fully, A fully)
  B pays D ₹300 (settles both fully)
→ Only 2 transactions instead of 3-4
```

Ye **classic "optimal account balancing" problem** hai — LeetCode 465 ka real-world production version. Directly connects to tumhare 700+ DSA background — interview mein bata sakte ho "maine ek known algorithmic problem ko real production ledger system mein apply kiya, with correctness tests for edge cases like circular debts and floating-point rounding."

Edge cases jo handle karne padenge (genuinely hard): - Floating point rounding errors (paisa

mismatch by ₹0.01) — fix: integers mein paise store karo (₹ ko paise mein convert karke store karo, display time pe convert back) - Circular debts (A→B→C→A) — greedy algorithm khud resolve kar deta hai net balance approach se - Partial settlements (user beech mein hi kuch settle kar de)

---

9. Proactive Insight Agent — Real Differentiation

Ye woh part hai jo isse “reactive ledger” se “agentic assistant” banata hai:

- **Overdue nudge**: `"Rahul ne 3 baar late settle kiya hai iss mahine — isse pehle confirm kar lo agla split dena hai ya nahi"`
- **Pattern insight**: `"Tumhara khaane ka kharcha iss mahine 20% zyada hai pichle mahine se"`
- **Inactivity nudge**: `"Group mein 10 din se koi expense nahi aaya — sab settle ho gaya kya?"`
- **Pre-emptive settle suggestion**: Mahine ke end mein khud message: `"Iss mahine ka final settle-up: A ko B se ₹450 lene hain. Confirm karu settlement mark karna?"`

Ye Celery Beat scheduled tasks se trigger hote hain, agent LLM call karke personalized message generate karta hai based on DB patterns — **agent khud initiate karta hai conversation, user pooche bina.** Yehi asli “agentic” hai, Splitwise jaise apps mein ye bilkul nahi hota.

---

10. Tech Stack

Layer	Technology
API	FastAPI + Uvicorn
Orchestration	LangGraph (StateGraph, tool-calling planner)
LLM	OpenRouter (DeepSeek/GPT-4o-mini) for reasoning, Groq (Llama 3.1) for response refining
Voice	OpenAI Whisper API
Vision/OCR	Vision LLM (bill/receipt parsing)
Database	PostgreSQL (async, source of truth)
ORM	SQLAlchemy 2.x async + Alembic migrations
Cache/State	Redis (Upstash) — pending confirmations, dedup, locks
Task Queue	Celery + Redis (proactive insight scheduling)
WhatsApp	Meta Cloud API
Deployment	Docker + Docker Compose, Render/Railway

---

11. Project Structure

```
KharchaSaathi/
├── app/
│   ├── agents/
│   │   ├── planner_agent.py        # LangGraph orchestrator
│   │   ├── intent_agent.py
│   │   ├── entity_resolver_agent.py
```

```
│   │   ├── insight_agent.py        # proactive nudges
│   │   └── refiner_agent.py
│   ├── algorithms/
│   │   └── debt_optimizer.py       # min-cash-flow, pure DSA, no LLM
│   ├── api/
│   │   ├── webhook.py
│   │   └── whatsapp_client.py
│   ├── db/
│   │   ├── models.py
│   │   ├── schema.sql
│   │   └── crud/
│   │       ├── expenses.py
│   │       ├── debts.py
│   │       └── settlements.py
│   ├── memory/
│   │   └── redis_store.py          # pending state, dedup, locks
│   ├── services/
│   │   ├── ocr_service.py
│   │   ├── whisper_service.py
│   │   └── whatsapp_service.py
│   ├── tools/                       # agent tool-calling functions
│   │   ├── add_expense_tool.py
│   │   ├── settle_tool.py
│   │   ├── query_balance_tool.py
│   │   └── undo_tool.py
│   └── core/
│       ├── config.py
│       └── logging.py
├── scheduler/
│   ├── celery_app.py
│   └── insight_tasks.py            # daily/weekly proactive nudges
├── tests/
│   ├── test_debt_optimizer.py      # edge cases: circular debt, rounding
│   └── test_agent_pipeline.py
├── alembic/
├── docker-compose.yml
├── Dockerfile
└── README.md
```

---

12. Scheduled Tasks (Celery Beat)

Task	Schedule	Purpose
`check_overdue_settlements`	Daily 9 PM IST	Long-pending debts ke liye nudge
`weekly_spending_insight`	Every Sunday	Spending pattern summary per group
`inactivity_check`	Every 3 days	Group inactive hai to check-in message
`month_end_settle_suggestion`	Last day of month	Proactive settle-up summary

---

13. Interview Talking Points — Kya Highlight Karna

1. **Design decision: LLM vs deterministic algorithm** — settlement calculation pure DSA hai (no hallucination risk), sirf language understanding LLM se hai. Ye judgment call hai jo dikhata hai “AI over-use” nahi kar rahe.

2. **Idempotency + at-least-once delivery handling** — distributed systems problem, real production concern.
3. **Ambiguity resolution via multi-turn state (Redis)** — stateless webhook ko stateful conversation mein convert karna.
4. **Debt-graph minimization** — known DSA problem (LeetCode 465 style) applied to real system, with correctness tests.
5. **Honest scoping** — core ledger idea Splitwise se naya nahi, differentiation UX + agentic layer + proactive intelligence mein hai. (Ye honestly bolna khud ek strong signal hai — interviewer ko pata chalta hai tum over-claim nahi karte.)

---

14. Build Order (Suggested)

1. Core Postgres schema + CRUD (no AI yet) — basic ledger working
2. Debt optimizer algorithm + tests (pure DSA, isolated, testable)
3. WhatsApp webhook + FastAPI skeleton
4. Intent + entity resolution agent (text only first)
5. Redis pending-confirmation loop (multi-turn ambiguity handling)
6. OCR + Voice input
7. Celery Beat proactive insight agent
8. Deploy + get 5-10 real users (hostel/friends) testing it for 2 weeks — real usage proof for resume